

l3draw 宏包

核心绘图支持

L^AT_EX Project*

Released 2025-06-30

1 l3draw 文档

l3draw 宏包提供了一套用于在 expl3 中创建（矢量）绘图的工具。它深受 TikZ 系统的 pgf 层的启发，许多接口具有相同的形式。然而，此处提供的代码完全基于核心 expl3 思想并使用 L^AT_EX3 FPU 进行数值支持。

除非另有说明，l3draw 中的数值表达式均作为浮点表达式处理。这意味着它们可以包含或省略显式单位。如果省略单位，它们将自动假定为以 (T_EX) 点为单位。

此处的代码是高度实验性的。

*E-mail: latex-team@latex-project.org

1.1 绘图

```
\draw_begin: \draw_begin:  
\draw_end:   ...
```

```
\draw_end:
```

每个绘图都应在 `\draw_begin:/\draw_end:` 函数对中创建。`begin` 函数为此处其余函数设置了许多关键数据结构：除非另有指定，否则在此“环境”之外使用 `\draw_...` 函数是不受支持的。

在环境中创建的绘图将由 `\draw_end:` 函数插入到排版流中，该函数将在需要时切换出垂直模式。



```
\dim_new:N \l_mypos_dim  
\draw_begin:  
  \draw_path_moveto:n { 0cm , \l_mypos_dim }  
  \draw_path_lineto:n { 1cm , \l_mypos_dim }  
  \dim_set:Nn \l_mypos_dim { 1cm }  
  \draw_path_lineto:n { 1cm , \l_mypos_dim }  
  \draw_path_close:  
  \draw_path_use_clear:n { stroke }  
\draw_end:
```

在绘图内部，只能出现直接用于绘图（来自 `l3draw`）的命令以及不产生任何排版输出的命令。例如，可以使用 `\int_step_function:nnnn` 或类似命令在绘图内部包含循环。另一方面，文本不应直接包含在绘图中，而应使用适当的 `l3draw` 命令插入。

绘图环境设置以下标准行为：

- 填充重叠的非零规则
- 线条的平头端点
- 线条连接的斜接，斜接因子为 10
- 实线描边

在绘图内部，有不同的方式引用位置。点的坐标是相对于当前绘图轴给出的。这些可以在代码级别进行操作和跟踪。其基础是〈画布〉，它在 `draw` 级别基本上是固定的，并与纸张对齐。最初，两组轴是重合的。（可以在驱动程序级别操作画布轴：这对于 `draw` 级别是“透明的”，因此只应在严格需要时使用。）

绘图的边界框是通过跟踪起始和结束之间 `draw` 命令的大小来确定的。边界框是（大致）包含绘图中提及的所有坐标的最小框。这可以包括自动生成的坐标，例如构建弧所需的支撑点。

`\draw_set_baseline:n \draw_set_baseline:n {<长度>}`

通常，绘图边界框的基线会自动计算在绘图底部。可以使用 `\draw_set_baseline:n` 函数调整此设置。如果绘图坐标导致 y 轴值低于<长度>，则绘图将具有深度和高度。

```
text
\draw_begin:
  \draw_path_rectangle:nn { 0 , 0 } { 2ex , 1ex }
  \draw_path_use:n { stroke }
\draw_end:
text
\draw_begin:
  \draw_path_rectangle:nn { 0 , 1ex } { 2ex , 1ex }
  \draw_set_baseline:n { 0pt }
  \draw_path_use:n { stroke }
\draw_end:
text
\draw_begin:
  \draw_path_rectangle:nn { 0 , -1ex } { 2ex , 1ex }
  \draw_set_baseline:n { -0.5ex }
  \draw_path_use:n { stroke }
\draw_end:
text
```

text□text□text□text

```
\draw_suspend_begin: \draw_suspend_begin:
\draw_suspend_end:   ...
```

```
\draw_suspend_end:
```

暂停所有绘图机制，以允许创建“正常”内容。通常，此环境将应用于可能包含嵌套图片的盒子内部。

```

\draw_begin:
\draw_path_moveto:n { 0cm , 0cm }
\hbox_set:Nn \l_tmpa_box
{
  \draw_suspend_begin:
  This~is~normal~text.
  \draw_begin: % A subpicture
  \draw_path_moveto:n { 1cm , 0cm }
  \draw_path_lineto:n { 1cm , 1cm }
  This is normal text. More text. \draw_path_use_clear:n { stroke }
  \draw_end:
  More~text.
  \draw_suspend_end:
}
\draw_box_use:N \l_tmpa_box
\draw_path_lineto:n { 0cm , 1cm }
\draw_path_use_clear:n { stroke }
\draw_end:

```

\g_draw_id_int 每个绘图都被赋予一个唯一的标识：每次使用 `\draw_begin:` 时，此标识都会递增。

1.2 设置图形状态

在绘图环境中，许多函数控制绘图的显示方式。请注意，这些函数都全局应用，尽管有些在每次绘图开始时（`\draw_begin:`）会重置。

```
\l_draw_default_linewidth_dim
```

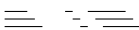
描边线宽的默认值，在每次绘图开始时（`\draw_begin:`）设置。

```
\draw_set_linewidth:n \draw_set_linewidth:n {⟨宽度⟩}
```

将描边使用的宽度设置为⟨宽度⟩（一个`⟨fp expr⟩`）。

<code>\draw_set_dash_pattern:nn</code>	<code>\draw_set_dash_pattern:nn {<模式>} {<相位>}</code>
--	--

指定虚线模式。〈模式〉本身是一个逗号分隔的条目列表，表示线条的“开”和“关”部分。这些都是〈fp expr〉，并根据需要重复。因此，〈模式〉可以具有任意长度。〈相位〉指定在第一个“开”线期间，模式应该从哪里开始。

	<pre> \draw_begin: \draw_set_dash_pattern:nn { 0.5cm , 0.5cm , 0.1cm , 0.2cm } { 0cm } \draw_path_moveto:n { 0cm , 0cm } \draw_path_lineto:n { 2cm , 0cm } \draw_path_use_clear:n { stroke } \draw_set_dash_pattern:nn { 0.5cm , 0.5cm , 0.1cm , 0.2cm } { 0.1cm } \draw_path_moveto:n { 0cm , 1mm } \draw_path_lineto:n { 2cm , 1mm } \draw_path_use_clear:n { stroke } \draw_set_dash_pattern:nn { 0.5cm , 0.5cm , 0.1cm , 0.2cm } { 0.2cm } \draw_path_moveto:n { 0cm , 2mm } \draw_path_lineto:n { 2cm , 2mm } \draw_path_use_clear:n { stroke } \draw_end: </pre>
	

设置空模式将产生实线。

请注意，此处的〈模式〉接口与 pgf 中的不同：列表是逗号分隔的，而不是用花括号分组的。

<code>\draw_set_nonzero_rule:</code>	<code>\draw_set_nonzero_rule:</code>
--------------------------------------	--------------------------------------

<code>\draw_set_evenodd_rule:</code>	分别激活非零缠绕数规则或奇偶规则，以确定填充或裁剪区域的内部。由于技术原因，这些命令不受作用域影响，并持续应用。
--------------------------------------	--

<code>\draw_set_cap_but:</code>	<code>\draw_set_cap_but:</code>
---------------------------------	---------------------------------

<code>\draw_set_cap_rectangle:</code>	将描边末端位置的样式设置为平头、矩形或圆形之一。
<code>\draw_set_cap_round:</code>	

<code>\draw_set_join_bevel:</code>	<code>\draw_set_join_miter:</code>
------------------------------------	------------------------------------

<code>\draw_set_join_miter:</code>	将描边连接的样式设置为斜角、斜接或圆形之一。
------------------------------------	------------------------

<code>\draw_set_join_round:</code>

`\draw_set_miterlimit:n \draw_set_miterlimit:n {⟨因子⟩}`

设置斜接连接的线条的斜接⟨因子⟩，如 PDF 和 PostScript 手册中所述。⟨因子⟩是一个 $\langle fp\ expr \rangle$ 。

1.3 作用域绘图元素

作用域绘图元素对于允许嵌套子部分是必要的。这些功能具有特定的使用要求：必须满足前置条件。特别是，每种绘图作用域也构成一个组（`\group_begin:/\group_end:` 对）：因此，它们必须正确嵌套。

```
\draw_scope_begin: \draw_scope_begin:
\draw_scope_end:   ...
```

```
\draw_scope_end:
```

为绘图内部的状态设置创建作用域。作用域形成一个 $\text{T}_{\text{E}}\text{X}$ 组，但也会局部化全局状态变量（例如 `\l_draw_default_linewidth_dim`），以及驱动程序级别的概念，例如线条的终止。

```

\draw_begin:
\draw_scope_begin:
\group_begin:
  \draw_set_linewidth:n { 2pt }
  \draw_path_rectangle:nn { 0 , 0 } { 2ex , 2ex }
  \draw_path_use:n { stroke }
\group_end:
\draw_path_rectangle:nn { 3ex , 0ex } { 2ex , 2ex }
\draw_path_use:n { stroke }
\draw_scope_end:
\draw_path_rectangle:nn { 6ex , 0ex } { 2ex , 2ex }
\draw_path_use_clear:n { stroke }
\draw_end:

```



受作用域限制的全局图形概念包括：

- 线宽
- 描边和填充颜色
- 虚线模式
- 线条连接和端点，包括斜接限制
- 裁剪路径
- 画布（驱动程序）变换

1.4 点

支持点（坐标）计算的函数是可扩展的，可以在绘图环境之外使用。所有点函数的输出都是元组。此输出形式随后适合作为后续点计算的输入，即，需要 $\langle$ 点 $\rangle$ 时，可以将其作为元组给出。这可能包括单位和周围的括号，例如

```
1,2
(1,2)
```

```

1cm,3pt
(1pt,2cm)
2 * sind(30), 2^4in

```

都是有效的输入形式。请注意，元组的每个部分本身都可以是浮点表达式。

点坐标是相对于画布轴的，但可以通过 `\draw_point_transform:n` 进行变换。这些操作由许多高级函数（例如路径构造）自动应用，允许绘图的某些部分进行旋转、缩放或倾斜。这发生在将任何数据写入驱动程序之前，因此此类操作由绘图机制跟踪。有关画布轴本身的后端级操作，请参见 `\@@_backend_cm:nnnn`。

请注意，与 `pgf` 相比，此处可以直接给出点的位置。

1.4.1 基本点函数

```

\draw_point_polar:nn * \draw_point_polar:nn {<半径>} {<角度>}
\draw_point_polar:nnn * \draw_point_polar:nnn {<半径-a>} {<半径-b>} {<角度>}

```

给出在<角度>（以度为单位的`<fp expr>`）和<半径>处的点的坐标。三参数版本接受两个不同长度的半径。

请注意，此处的接口与 `pgf` 中的有所不同：`l3draw` 中的单半径和双半径版本使用单独的函数，而 `pgf` 中它们使用相同的函数和一个关键字。

```

\draw_point_unit_vector:n * \draw_point_unit_vector:n {<点>}

```

展开为从原点指向<点>方向的单位向量坐标。如果<点>位于原点，则返回一个垂直单位向量。

```

\draw_point_transform:n * \draw_point_transform:n {<点>}

```

根据当前变换矩阵评估<点>的位置。此操作由大多数高级函数（例如路径操作）自动应用。

1.4.2 基于向量基的点

除了给出明确的值，还可以根据底层方向向量来描述点。后者最初与标准笛卡尔轴重合，但可以由用户更改。

```

\draw_xvec:n \draw_xvec:n {<点>}

```

```

\draw_yvec:n 定义适当的基向量以指向画布上的<点>。 $x$  和  $y$  向量的标准设置是沿相关画布轴
\draw_zvec:n 1 cm，而  $z$  向量则取适当的方向。

```

```
\draw_point_vec:nn * \draw_point_vec:nn {⟨x 缩放⟩} {⟨y 缩放⟩}
\draw_point_vec:nnn * \draw_point_vec:nnn {⟨x 缩放⟩} {⟨y 缩放⟩} {⟨z 缩放⟩}
```

展开为点坐标，该点坐标是 x 向量的⟨ x 缩放⟩倍和 y 向量的⟨ y 缩放⟩倍。三参数版本将其扩展为包含 z 向量。

```
\draw_point_vec_polar:nn * \draw_point_vec_polar:nn {⟨半径⟩} {⟨角度⟩}
\draw_point_vec_polar:nnn * \draw_point_vec_polar:nnn {⟨半径-a⟩} {⟨半径-b⟩} {⟨角度⟩}
```

给出在⟨角度⟩（以度为单位的⟨ $fp\ expr$ ⟩）和⟨半径⟩处的点的坐标，相对于当前 x 和 y 向量。三参数版本接受两个不同长度的半径。

请注意，此处的接口与 `pgf` 中的有所不同：`l3draw` 中的单半径和双半径版本使用单独的函数，而 `pgf` 中它们使用相同的函数和一个关键字。

1.4.3 交点

```
\draw_point_intersect_lines:nnnn * \draw_point_intersect_lines:nnnn {⟨点 1⟩} {⟨点 2⟩} {⟨点 3⟩} {⟨点 4⟩}
```

评估一条连接⟨点 1⟩和⟨点 2⟩的直线与第二条连接⟨点 3⟩和⟨点 4⟩的直线的交点。如果直线不相交或重合，则会发生错误。

```
\draw_point_intersect_circles:nnnnn * \draw_point_intersect_circles:nnnnn
{⟨中心 1⟩} {⟨半径 1⟩} {⟨中心 2⟩} {⟨半径 2⟩} {⟨根⟩}
```

评估一个以⟨中心 1⟩和⟨半径 1⟩为圆心半径的圆与第二个以⟨中心 2⟩和⟨半径 2⟩为圆心半径的圆的交点。如果圆不相交或重合，则会发生错误。

请注意，此处的接口与 `pgf` 中的参数顺序不同，`pgf` 中是两个中心然后是两个半径。

```
\draw_point_intersect_line_circle:nnnnn * \draw_point_intersect_line_circle:nnnnn
{⟨点 1⟩} {⟨点 2⟩} {⟨中心⟩} {⟨半径⟩} {⟨根⟩}
```

评估一条连接⟨点 1⟩和⟨点 2⟩的直线与一个以⟨中心⟩和⟨半径⟩为圆心半径的圆的交点。如果直线和圆不相交，则会发生错误。

1.4.4 插值

```
\draw_point_interpolate_line:nnn * \draw_point_interpolate_line:nnn {⟨部分⟩} {⟨点 1⟩} {⟨点 2⟩}
```

展开为连接⟨点 1⟩和⟨点 2⟩的直线上⟨部分⟩处的点。⟨部分⟩可以是内插或外推，是一个表示直线上百分比的浮点值，例如，值 0.5 表示在两点之间的一半。

```
\draw_point_interpolate_distance:nnn * \draw_point_interpolate_distance:nnn {⟨距离⟩} {⟨点表达式 1⟩} {⟨点表达式 2⟩}
```

展开为连接⟨点 1⟩和⟨点 2⟩的直线上⟨距离⟩处的点。⟨距离⟩可以是内插或外推。

```
\draw_point_interpolate_curve:nnnnnn * \draw_point_interpolate_curve:nnnnnn {⟨部分⟩}  
{⟨起点⟩} {⟨控制点 1⟩} {⟨控制点 2⟩} {⟨终点⟩}
```

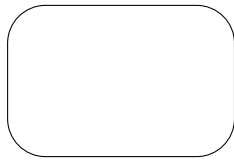
展开为在⟨起点⟩和⟨终点⟩之间，由⟨控制点 1⟩和⟨控制点 2⟩定义的曲线上⟨部分⟩处的点。⟨部分⟩可以是内插或外推，是一个表示曲线上百分比的浮点值，例如，值 0.5 表示在曲线的一半处。

1.5 路径

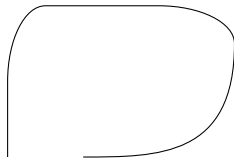
路径是通过组合一个或多个操作，然后应用一个或多个动作来构建的。因此，在路径被“使用”之前，它可以被操作，甚至完全丢弃。任何时候只有一个路径处于活动状态，并且路径不受 TeX 分组的影响。

`\draw_path_corner_arc:nn` `\draw_path_corner_arc:nn {<长度 1>} {<长度 2>}`

设置应用于路径拐角的圆角程度：两个<长度>值是曲线应从拐角开始的距离。第一个<长度>应用于路径“进入”拐角的部分（即从上一个路径操作），第二个应用于“离开”拐角的部分。如果两个值都为 0pt，则拐角将不进行圆角处理。这些值在当前 T_EX 组的作用域内应用。

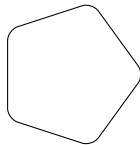


```
\draw_begin:
\draw_path_corner_arc:nn { 5mm } { 5mm }
\draw_path_rectangle_corners:nn
{ 0cm , 0cm } { 3cm , 2cm }
\draw_path_use_clear:n { stroke }
\draw_end:
```



```
\draw_begin:
\draw_path_corner_arc:nn { 10mm } { 5mm }
\draw_path_moveto:n { 0cm , 0cm }
\draw_path_lineto:n { 0cm , 2cm }
\draw_path_lineto:n { 3cm , 2cm }
\draw_path_curveto:nnn
{ 3cm , 0cm } { 2cm , 0cm } { 1cm , 0cm }
\draw_path_use_clear:n { stroke }
\draw_end:
```

创建的拐角对于精确直角是四分之一圆，否则是近似值。闭合路径将导致正确圆角。



```
\draw_begin:
\draw_path_corner_arc:nn { 4pt } { 4pt }
\draw_path_moveto:n
{ \draw_point_polar:nn { 1cm } { 0 } }
\int_step_inline:nnnn { 72 } { 72 } { 359 }
{
\draw_path_lineto:n
{ \draw_point_polar:nn { 1cm } { #1 } }
}
\draw_path_close:
\draw_path_use_clear:n { stroke }
\draw_end:
```

`\draw_path_close:` `\draw_path_close:`

通过从当前点到路径的起点追加一条直线来闭合路径的当前部分。通常，任何要填充的路径都应包含闭合指令。

`\draw_path_use:n` `\draw_path_use:n {⟨动作 (们)⟩}`

`\draw_path_use_clear:n` 插入当前路径，执行一个或多个可能的⟨动作⟩（逗号分隔列表）：

- `stroke` 沿当前路径绘制一条线
- `draw stroke` 的同义词
- `fill` 用当前文件颜色填充路径内部
- `clip` 裁剪路径外部的任何内容

动作按给定顺序应用，与输入顺序无关。请注意，虽然可以在不清除路径的情况下使用路径，但通常的使用情况是清除路径（这会重置宏级别的数据结构）。

`\draw_path_replace_bb:` `\draw_path_replace_bb:`

用当前路径指定的边界框替换绘图的当前边界框：即使 `\l_draw_bb_update_bool` 为 `false`，此操作也会应用。然后清除当前路径。请注意，此函数不更改 `\l_draw_bb_update_bool`：用户可能希望将其设置为 `false`，以便边界框在后续操作中保持不变。

1.5.1 绘图轴上的路径操作

标准路径函数都受活动变换矩阵的影响，即相对于绘图轴而不是画布进行工作。

`\draw_path_moveto:n` `\draw_path_moveto:n {⟨点⟩}`

将路径的参考点移动到⟨点⟩，但不会将其与任何前一个点连接。

`\draw_path_lineto:n \draw_path_lineto:n {⟨点⟩}`

用直线将当前路径连接到⟨点⟩。通常，为了查看器可靠处理，`\draw_path_moveto:n`操作应先于路径上首次使用 `\draw_path_lineto:n`。



```
\draw_begin:
\draw_path_moveto:n { 0cm , 0cm }
\draw_path_lineto:n { 1cm , 1cm }
\draw_path_lineto:n { 2cm , 1cm }
\draw_path_lineto:n { 3cm , 0.5cm }
\draw_path_lineto:n { 3cm , 0cm }
\color_fill:n { yellow!80!black }
\draw_path_use_clear:n { fill , stroke }
\draw_end:
```



```
\draw_begin:
\draw_path_moveto:n { 0cm , 0cm }
\draw_path_lineto:n { 1cm , 1cm }
\draw_path_lineto:n { 2cm , 1cm }
\draw_path_moveto:n { 2cm , 1cm } % Begins a new part
\draw_path_lineto:n { 3cm , 0.5cm }
\draw_path_lineto:n { 3cm , 0cm }
\color_fill:n { yellow!80!black }
\draw_path_use_clear:n { fill , stroke }
\draw_end:
```

`\draw_path_curveto:nnn \draw_path_curveto:nnn {⟨控制点 1⟩} {⟨控制点 2⟩} {⟨终点⟩}`

用由三次贝塞尔点⟨控制点 1⟩和⟨控制点 2⟩定义的曲线将当前路径连接到⟨终点⟩。路径（和图像）的边界框将完全包含曲线和控制点，即它可能比仅包含曲线本身所需的尺寸更大。



```
\draw_begin:
\draw_path_moveto:n { 0cm , 0cm }
\draw_path_curveto:nnn
{ 1cm , 1cm } % 第一个控制点
{ 2cm , 1cm } % 第二个控制点
{ 3cm , 0cm } % 终点
\color_fill:n { yellow!80!black }
\draw_path_use_clear:n { fill , stroke }
\draw_end:
```

`\draw_path_curveto:nn` `\draw_path_curveto:nn {<控制点>} {<终点>}`

用由二次贝塞尔点<控制点>定义的曲线将当前路径连接到<终点>。路径（和图像）的边界框将完全包含曲线和计算出的（三次）控制点，即它可能比仅包含曲线本身所需的尺寸更大。



```

\draw_begin:
  \draw_path_moveto:n { 0cm , 0cm }
  \draw_path_curveto:nn
    { 1cm , 1cm }
    { 2cm , 0cm }
  \color_fill:n { yellow!80!black }
  \draw_path_use_clear:n { fill , stroke }
\draw_end:

```

`\draw_path_arc:nnn` `\draw_path_arc:nnn {<角度 1>} {<角度 2>} {<半径>}`
`\draw_path_arc:nnnn` `\draw_path_arc:nnnn {<角度 1>} {<角度 2>} {<半径-a>} {<半径-b>}`

将当前路径与<角度 1>和<角度 2>之间，半径为<半径>的弧线连接。四参数版本接受两个不同长度的半径。



```

\draw_begin:
  \draw_path_moveto:n { 0cm , 0cm }
  \draw_path_lineto:n { 0cm , 1cm }
  \draw_path_arc:nnn { 180 } { 90 } { 0.5cm }
  \draw_path_lineto:n { 3cm , 1.5cm }
  \draw_path_arc:nnn { 90 } { -45 } { 0.5cm }
  \draw_path_use_clear:n { fill }
\draw_end:

```

请注意，此处的接口与 `pgf` 中的参数顺序不同，`pgf` 中是两个中心然后是两个半径。



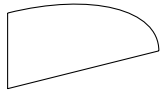
```

\draw_begin:
  \draw_path_moveto:n { 0cm , 0cm }
  \draw_path_arc:nnnn { 180 } { 45 } { 2cm } { 1cm }
  \draw_path_use_clear:n { stroke }
\draw_end:

```

`\draw_path_arc_axes:nnnn` `\draw_path_arc_axes:nnn` {<角度 1>} {<角度 2>} {<向量 1>} {<向量 2>}

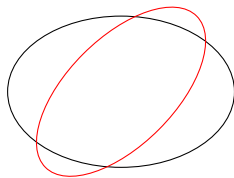
将从<角度 1>到<角度 2>的椭圆部分，其轴线沿<向量 1>和<向量 2>，追加到当前路径。



```
\draw_begin:
  \draw_path_moveto:n { 0cm , 0cm }
  \draw_path_lineto:n { 2cm , 5mm }
  \draw_path_moveto:n { 0cm , 0cm }
  \draw_path_lineto:n { 0cm , 1cm }
  \draw_path_moveto:n { 2cm , 5mm }
  \draw_path_arc_axes:nnnn { 0 } { 90 }
    { 2cm , 5mm } { 0cm , 1cm }
  \draw_path_use_clear:n { stroke }
\draw_end:
```

`\draw_path_ellipse:nnn` `\draw_path_ellipse:nnn` {<中心>} {<向量 1>} {<向量 2>}

将以<中心>为圆心，轴线沿<向量 1>和<向量 2>的椭圆追加到当前路径。如果路径已经非空，则开始新部分。请注意，底层绘图是由带适当移动的弧线构建的：接口是一种更高效的便利。



```
\draw_begin:
  \draw_path_ellipse:nnn
    { 1cm , 0cm }
    { 1.5cm , 0cm }
    { 0cm , 1cm }
  \draw_path_use_clear:n { stroke }
  \color_select:n { red }
  \draw_path_ellipse:nnn
    { 1cm , 0cm }
    { 1cm , 1cm }
    { -0.5cm , 0.5cm }
  \draw_path_use_clear:n { stroke }
\draw_end:
```

请注意，任何变换都应用于完成的椭圆，而不是轴。

`\draw_path_circle:nn` `\draw_path_circle:nn` {<中心>} {<半径>}

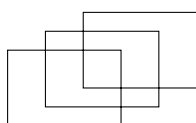
将以<中心>为圆心，半径为<半径>的圆追加到当前路径。这是 `\draw_path_ellipse:nnn` 的快捷方式。

```

\draw_path_rectangle:nn          \draw_path_rectangle:nn {<左下角>} {<位移>}
\draw_path_rectangle_corners:nn \draw_path_rectangle_corners:nn {<左下角>} {<右上角>}

```

将从<左下角>开始的矩形追加到当前路径，矩形的大小由<位移>或<右上角>的位置确定。



```

\draw_begin:
  \draw_path_rectangle:nn
    { 1cm , 0cm }
    { 1.5cm , 1cm }
  \draw_path_rectangle:nn
    { 1.5cm , 0.25cm }
    { 1.5cm , 1cm }
  \draw_path_rectangle:nn
    { 2cm , 0.5cm }
    { 1.5cm , 1cm }
  \draw_path_use_clear:n { draw }
\draw_end:

```



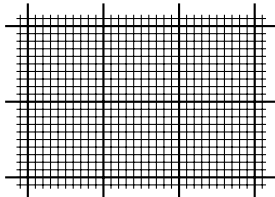
```

\draw_begin:
  \draw_path_rectangle_corners:nn
    { 1cm , 0cm }
    { 1.5cm , 1cm }
  \draw_path_use_clear:n { draw }
\draw_end:

```

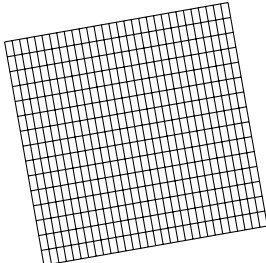
```
\draw_path_grid:nnnn \draw_path_grid:nnnn {<x 步长>} {<y 步长>} {<左下角>} {<右上角>}
```

在由<左下角>和<右上角>定义的矩形内部构建一个<x 步长>和<y 步长>的网格，并将其追加到当前路径。网格将对齐，使网格线通过原点，如果起始/结束位置不完全对齐，可能会导致“突出”的末端。



```
\draw_begin:
\draw_set_linewidth:n { 0.8pt }
\draw_path_grid:nnnn
{ 1cm } { 1cm }
{ -3mm , -3mm }
{ 33mm , 23mm }
\draw_path_use_clear:n { stroke }
\draw_set_linewidth:n { 0.4pt }
\draw_path_grid:nnnn
{ 1mm } { 1mm }
{ -1.5mm , -1.5mm }
{ 31.5mm , 21.5mm }
\draw_path_use_clear:n { stroke }
\draw_end:
```

任何变换都应用于完成的网格。



```
\draw_begin:
\draw_transform_rotate:n { 10 }
\draw_path_grid:nnnn
{ 1mm } { 2mm }
{ 0mm , 0mm }
{ 30mm , 30mm }
\draw_path_use_clear:n { stroke }
\draw_end:
```

1.5.2 路径作用域

```
\draw_path_scope_begin: \draw_path_scope_begin:
\draw_path_scope_end:   ...
\draw_path_scope_end:
```

暂停（并保存）当前（部分）路径，在作用域内初始化新路径。路径操作仅在使用时才写入输出，因此作用域路径存储在 expl3 级别，而不是输出中。

1.5.3 画布轴上的路径操作

对于专业工作，提供了一些相对于画布轴工作的函数，即这些函数忽略变换矩阵。

<code>\draw_path_canvas_moveto:n</code>	<code>\draw_path_canvas_moveto:n {⟨画布点⟩}</code>
---	---

将路径的参考点移动到⟨画布点⟩，但不会将其与任何前一个点连接。

<code>\draw_path_canvas_lineto:n</code>	<code>\draw_path_canvas_lineto:n {⟨画布点⟩}</code>
---	---

用直线将当前路径连接到⟨画布点⟩。

<code>\draw_path_canvas_curveto:nnn</code>	<code>\draw_path_canvas_curveto:nnn {⟨控制点 1⟩} {⟨控制点 2⟩} {⟨终点⟩}</code>
--	---

用由三次贝塞尔点⟨控制点 1⟩和⟨控制点 2⟩定义的曲线将当前路径连接到⟨终点⟩。这些位置作为画布点给出。

1.6 边界框

<code>\g_draw_bb_xmin_dim</code>	自动计算边界框的角点值。请注意，这是绘图的表观尺寸：材料可以放置在边界框之外，请参见 <code>\l_draw_bb_update_bool</code> 。
<code>\g_draw_bb_ymin_dim</code>	
<code>\g_draw_bb_xmax_dim</code>	
<code>\g_draw_bb_ymax_dim</code>	

<code>\l_draw_bb_update_bool</code>	所有函数都会自动更新图像的边界框，除非另有指定。此行为可通过 <code>\l_draw_bb_update_bool</code> 布尔值选择。
-------------------------------------	---

在 L^AT_EX 中，每个绘图原点的绝对位置被记录为一个属性：属性 `xpos`、`ypos` 和 `abspace` 被保存。绘图被标识为 `draw.⟨id⟩`，其中 `⟨id⟩` 是 `\g_draw_id_int` 的值。

1.7 盒子和箱子

```
\draw_box_use:N \draw_box_use:N <盒子>
\draw_box_use:Nn \draw_box_use:Nn <盒子> {(点)}
```

将<盒子>插入绘图中，考虑当前的变换矩阵和偏移，如果边界框处于活动状态（参见 `\l_draw_bb_update_bool`），则调整绘图边界框以包含盒子的（表观）尺寸。

```

\draw_begin:
  \draw_path_moveto:n { 0cm , 0cm }
  \draw_path_lineto:n { 0cm , 1cm }
  \draw_path_use_clear:n { stroke }
  \hbox_set:Nn \l_tmpa_box
    { This~is~text. }
  \draw_box_use:N \l_tmpa_box
\draw_end:

\draw_begin:
  \draw_transform_matrix_absolute:nnnn { 2 } { 0 } { 1 } { 2 }
  \draw_path_moveto:n { 0cm , 0cm }
  \draw_path_lineto:n { 0cm , 1cm }
  \draw_path_use_clear:n { stroke }
  \hbox_set:Nn \l_tmpa_box
    { This~is~text. }
  \draw_box_use:N \l_tmpa_box
\draw_end:

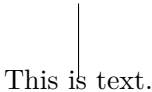
```

This is text.

如果给出了<点>，则盒子的参考点放置在那里：否则它位于绘图的原点。

<code>\draw_coffin_use:Nnn</code>	<code>\draw_coffin_use:Nnn</code> $\langle$ 箱子 $\rangle$ $\{ \langle$ 水平极点 $\rangle \}$ $\{ \langle$ 垂直极点 $\rangle \}$
<code>\draw_coffin_use:Nnnn</code>	<code>\draw_coffin_use:Nnnn</code> $\langle$ 箱子 $\rangle$ $\{ \langle$ 水平极点 $\rangle \}$ $\{ \langle$ 垂直极点 $\rangle \}$ $\{ \langle$ 点 $\rangle \}$

将 $\langle$ 箱子 $\rangle$ 插入绘图中，考虑当前的变换矩阵和偏移，如果边界框处于活动状态（参见 `\l_draw_bb_update_bool`），则调整绘图边界框以包含箱子的（表观）尺寸。箱子与原点的对齐点由 $\langle$ 水平极点 $\rangle$ 和 $\langle$ 垂直极点 $\rangle$ 的交点指定。

	<code>\draw_begin:</code>
	<code>\draw_path_moveto:n { 0cm , 0cm }</code>
	<code>\draw_path_lineto:n { 0cm , 1cm }</code>
	<code>\draw_path_use_clear:n { stroke }</code>
	<code>\hcoffin_set:Nn \l_tmpa_coffin</code> <code>{ This~is~text. }</code>
	<code>\draw_coffin_use:Nnn \l_tmpa_coffin { hc } { vc }</code>
	<code>\draw_end:</code>

如果给出了 $\langle$ 点 $\rangle$ ，则箱子的极点交点放置在那里：否则它位于绘图的原点。

1.8 变换

点通常相对于画布轴保持不变使用。这可以通过应用变换矩阵进行修改。画布轴本身可以使用 `\@@_backend_cm:nnnn` 进行调整：请注意，这对于绘图代码是透明的，因此不进行跟踪。

<code>\draw_transform_matrix:nnnn</code>	<code>\draw_transform_matrix:nnnn</code>
<code>\draw_transform_matrix_absolute:nnnn</code>	$\{ \langle a \rangle \}$ $\{ \langle b \rangle \}$ $\{ \langle c \rangle \}$ $\{ \langle d \rangle \}$

应用变换矩阵 $[\langle a \rangle \langle b \rangle \langle c \rangle \langle d \rangle]$ 。基本版本在现有变换的基础上应用变换；**absolute** 版本覆盖任何现有变换。此赋值是局部的。

<code>\draw_transform_shift:n</code>	<code>\draw_transform_shift:n</code> $\{ \langle$ 向量 $\rangle \}$
<code>\draw_transform_shift_absolute:n</code>	

将变换 $\langle$ 向量 $\rangle$ 应用于点。基本版本在现有向量的基础上应用向量；**absolute** 版本覆盖任何现有向量。任何现有变换矩阵在每次调整时都应用于偏移。此赋值是局部的。

<code>\draw_transform_triangle:nnn</code>	<code>\draw_transform_triangle:nnn</code>
	$\{ \langle$ 原点 $\rangle \}$ $\{ \langle$ 点 1 $\rangle \}$ $\{ \langle$ 点 2 $\rangle \}$

应用变换，使得坐标 $(0,0)$ 、 $(1,0)$ 和 $(0,1)$ 分别由 $\langle$ 原点 $\rangle$ 、 $\langle$ 点 1 $\rangle$ 和 $\langle$ 点 2 $\rangle$ 给出。此赋值是局部的。

<code>\draw_transform_rotate:n</code>	<code>\draw_transform_rotate:n</code> $\{ \langle$ 角度 $\rangle \}$
---------------------------------------	--

应用按 $\langle$ 角度 $\rangle$ 旋转，以度为单位逆时针测量。此旋转是任何现有变换的附加旋转。此赋值是局部的。

<code>\draw_transform_scale:n</code>	<code>\draw_transform_scale:n {⟨缩放⟩}</code>
<code>\draw_transform_xscale:n</code>	在 x 或 y (或两者) 方向应用⟨缩放⟩。此缩放是任何现有变换的附加缩放。此赋值是局部的。
<code>\draw_transform_yscale:n</code>	
<code>\draw_transform_xshift:n</code>	<code>\draw_transform_xshift:n {⟨x 偏移⟩}</code>
<code>\draw_transform_yshift:n</code>	应用适当的⟨ x 偏移⟩或⟨ y 偏移⟩。此偏移是任何现有偏移的附加偏移。此赋值是局部的。
<code>\draw_transform_xslant:n</code>	<code>\draw_transform_xslant:n {⟨倾斜⟩}</code>
<code>\draw_transform_yslant:n</code>	在 x 或 y 方向应用⟨倾斜⟩ (一个因子)。此倾斜是任何现有变换的附加倾斜。此赋值是局部的。
<code>\draw_transform_matrix_invert:</code>	<code>\draw_transform_matrix_invert:</code>
<code>\draw_transform_shift_invert:</code>	
	反转当前变换矩阵或偏移向量，视情况而定。此赋值是局部的。
<code>\draw_transform_matrix_reset:</code>	<code>\draw_transform_matrix_reset:</code>
<code>\draw_transform_shift_reset:</code>	
	重置当前变换矩阵或偏移向量，视情况而定。此赋值是局部的。

1.9 图层

绘图图层可用于改变元素堆叠的方式。特别是，当一个元素的性质依赖于另一个元素，但它需要位于其“父”元素之后时，图层非常有用。一个经典的例子是填充背景：它需要知道其后面材料的大小。

所有绘图都具有一个名为 `main` 的图层。此图层应始终存在，并且是添加“非图层”内容的图层。

<code>\draw_layer_new:n</code>	<code>\draw_layer_new:n {⟨图层⟩}</code>
	创建一个新的⟨图层⟩，然后可以在绘图中使用。 <code>main</code> 图层是预定义的。
<code>\draw_layer_begin:n</code>	<code>\draw_layer_begin:n {⟨图层⟩}</code>
<code>\draw_layer_end:</code>	...
	<code>\draw_layer_end:</code>
	开始和结束收集⟨图层⟩的材料，该图层应已创建 (且不能是 <code>main</code>)。为图层收集的材料在当前绘图的全局范围内可用。
<code>\l_draw_layers_clist</code>	活动图层列表：可在 <code>\draw_end:</code> 之前的任何位置给出。 <code>main</code> 图层应始终包含在内。